# AI assignment 1 report

**Haneen Osama – 9580**

**Basmala Mohamed – 9674**

**Rokaya Mohamed – 9582**

**Group 1 – Section 1**

***8 Puzzle Search Algorithms***

# AI ASSIGNMENT 1 REPORT

## Overview

We're working with an instance of a board as the initial state. The main idea is moving the blank, represented by 0 either horizontally or vertically and by using the different search algorithm, the goal state should be found with the path to it. Our goal state is the numbers in ascending order: 0,1,2,3,4,5,6,7,8

## 1. Our structure

Firstly, we have our main class, **Node**. Each board will be an instance of the node.

The node has different parameters, like the parent, the board itself, the move it made, the depth so far along with the cost. It also includes our tie-breaker which will be explained later.

*The functions used for almost all search algorithms:*

**swap:** To swap the two specified places and return a tuple of the board

**get_path:** It will take the goal node if found and backtrack till it finds the root. The root is the only node without a parent. The function will print the path and then use the reverse() function to get the path from root to goal instead of goal to parent.

**moveBlank:** Simply, it'll call the swap to move my Blank.

**print_board:** Despite the animation, this will print the board itself at the instance with the given parameters.

**print_solution:** Responsible to print the final solution, *not the animation*, it'll get the nodes from goal → parent, then reverse to get parent → goal. It will print the initial board, then it'll start printing what each move was made by each step, and the board given. This function is our **main printing** function, despite the animation.

## 2. Priorities and Notes:

There should be some sort of priority to decide which next move will be taken, despite the different data structures used. Our moves priorities are:

UP, DOWN, LEFT, RIGHT

According to the reference by the end of the assignment.

All nodes when running will share a few of the main functions to be able to print the path, the cost, the nodes expanded, search depth and running time.

To print path, use the function **get_path**

To print cost, get the len(path)

To print the nodes expanded, print the **expanded**

To print the depth, print the **depth**

***Note:*** depth and expanded are values each search algorithm return, along with goal_node to check if the search algorithm reached a goal or not.

For the time, simply use time.perf_counter() before starting the algorithm and time.perf_counter() – t0 right after finishing the search to find the running time.

**The tie-breaker:**

In nodes, when using letters or numbers, the tie breaker would simply be either alphabetically or ascendingly. But here, we're working with *boards*, so how would we decide who's first if two of same priorities?

Simply, we check who came first.

In the Node class,

```
self.order = next(Node._counter)   # creation order
```

this snipper of a code handle the creation of the node, deciding who came first. When there's a tie, this snippet works:

```
def __lt__(self, other):
    return (self.cost, self.order) < (other.cost, other.order)
```

It first decides by cost, if equal it'll check who came first.

## 3. Uninformed search algorithms

### 3.1 BFS (Breadth-First Search) Algorithm:

BFS is an uninformed search algorithm. Unlike informed search, it doesn't use any heuristic function or additional information about how close a board is to the goal. Instead, it explores the search space level by level.

BFS uses a **Queue** as its main data structure. A queue follows the FIFO (First In, First Out) principle, meaning that the first node added to the queue is the first one to be removed.

#### 3.1.1 The BFS main algorithm:

We start with two empty sets: frontier and explored.

For BFS, the data structures used are:

1. **Queue:** Stores the nodes waiting to be explored.
2. **Explored:** Stores the boards that have already been expanded, to avoid repeating the same states.

We first add the starting board to the frontier (queue).

While the frontier is not empty, the loop continues. If it becomes empty before finding the goal, the algorithm returns that no solution was found.

Considering the frontier still has values:

- Get the next node to be expanded. In BFS, this is the node that entered the queue first.
- Check if the board is the goal state. If yes, the algorithm stops and returns the solution path.
- If it is not the goal, continue the loop.
- If the board was already explored, skip it and get the next board.

If the board was not explored before, we add it to the explored set.

### 3.1.2 Why BFS?

Since every move in the 8-puzzle has the same cost (equal to 1), BFS guarantees finding the shortest path to the goal state, if a solution exists.

This is because BFS explores all possible states at depth 1 before depth 2, all states at depth 2 before depth 3, and so on.

However, BFS can consume a large amount of memory because it stores all the nodes waiting in the frontier, especially when the search reaches deeper levels.

---

### 3.2 DFS (Depth-First Search) Algorithm:

DFS is also an uninformed search algorithm. Similar to BFS, it does not use a heuristic function to guide its search.

However, unlike BFS, DFS explores one path as deeply as possible before going back and trying another path.

DFS uses a **Stack** as its main data structure. A stack follows the LIFO (Last In, First Out) principle, meaning that the last node added to the stack is the first one to be removed.

### 3.2.1 The DFS main algorithm:

We start with two empty sets: frontier and explored.

For DFS, the data structures used are:

1. **Stack:** Stores the nodes waiting to be explored.
2. **Explored:** Stores the boards that have already been expanded, to avoid visiting the same states repeatedly.

We first add the starting board to the frontier (stack).

While the frontier is not empty, the loop continues. If it becomes empty before finding the goal, the algorithm returns that no solution was found.

Considering the frontier still has values:

- Get the next node to be expanded. In DFS, this is the node that entered the stack last.
- Check if the board is the goal state. If yes, the algorithm stops and returns the solution path along with the extra parameters.
- If it is not the goal, continue the loop.
- If the board was already explored, skip it and get the next board.

If the board was not explored before, we add it to the explored set.

### 3.2.2 Why DFS?

DFS generally requires less memory than BFS because it focuses on exploring one path at a time instead of storing an entire level of nodes.

However, DFS does not guarantee finding the shortest path, even when a solution exists. It may find a solution that requires more moves than necessary because it focuses on depth rather than the number of moves needed to reach the goal.

In our 8-puzzle problem, all moves have the same cost (equal to 1), but DFS does not consider this cost when choosing which node to explore next.

### 3.3 IDS Algorithm

IDS algorithm runs DFS on different depths starting at an initial value and increasing the depth until it reaches the maximum value, this combines the low memory usage of DFS with the optimality of BFS, because the limit increases by only one each time it will always reach the goal at its shallowest depth.

### 3.3.1 Data Structures:

frontier is a stack and explored is a dictionary that maps each board with the lowest depth reached , if the same board is reached at a lower depth, it gets updated to find the most optimal solution , the dictionary resets at the start of each depth iteration.

**The Stack (Frontier ):**

| 8 |
|---|
| 3 |
| 1 |

**The Dictionary (Explored):**

| Key: | Depth: |
|---|---|
| {0,1,2,3,4,5,6,7,8} | 1 |
| {1,2,3,0,4,5,6,7,8} | 2 |

### 3.3.2 Algorithm:

Start at 0 then add 1 until reaching max depth, with each iteration explored is reset , while frontier is not empty it checks if the board is the goal, it returns the node. If the board is in the explored but current is at a less depth it updates the board in the dictionary, then it takes all the children and adds it in the Frontier.

### Pseudocode:

```
for limit = 0 to maxDepth: explored ={} #
        dictionary Frontier =stack()
        Frontier.append(startNode) while
        Frontier not empty:

                current= Frontier.pop() if
                current.board = goal:

                        return current

                if current.board in explored and explored[current.board] <= current.depth: continue

                explored[current.board] = current.depth # create new
                node

                newboard = moveBlank(current.board,move,blank)

                newNode = Node(newboard,current, move,current.depth + 1) Frontier.append(new_node)

return None
```

## 4. A* search algorithm:

The A* algorithm unlike the DFS, BFS or IDS is an informed search algorithm. It is aware of the environment around it and use it to get better results.

The A* use a heuristic function, there were two ways:

1. Manhattan Distance
2. Euclidean Distance

The only difference between them is their equation itself but other than that, the A* works the same for both.

For Manhattan as an example:

h = abs(current cell:x - goal:x) + abs(current cell:y - goal:y)

For Euclidean example:

h = sqrt((current cell:x - goal:x)2 + (current cell.y - goal:y)2)

A* = g + h, where g is 'How many moves so far?' and h is the heuristic.

### *4.1. The heuristic function:*

I take as an argument for the function the starting board's node. Each *tile* is compared by how far it is from the goal, and then add the values, that's our h. The g itself is then added as well.

The i and t used:

The i represent the grid itself:

| | | |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

While the t, is what value in that position.

| | | |
|---|---|---|
| 8 | 0 | 6 |
| 5 | 4 | 7 |
| 2 | 3 | 1 |

For this example:

t = 8 is at position i = 0

To be able to calculate the heuristic function, we need two nodes: Current, and goal.

Current = divmod( i , 3) ‘I’m at i, where is that on the grid?’

Goal = divmod( t, 3) ‘Where is tile t at?’

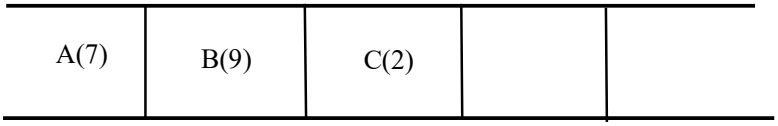
So divmod(4,3) means, “I’m at position 4, what are my rows and coloumns?”

They return the x and y, rows and coloumns, and we use these values to calculate either the Manahattan or Euclidean distance.

### *4.1.2 The A* main algorithm*

We start by frontier and explored, both are empty sets. For the A*, the data structure used is *Priority Queue*.

***The priority Queue:***




The priority queue is a queue but doesn’t particularly rely on the FIFO algorithm. It focuses on the cost itself, whoever is with lowest cost, will get out first.

**What if they’re both with the same cost?**

In normal problems, we solved the tie by choosing alphabetically. In our cases, they’re not letters they’re *boards*, so we made an edit to the priority queue.

*If there’s a tie, see who came first.*

A priority queue implemented with heapq stores entries as (cost, insertion_counter, node). The insertion counter acts as a tie-breaker, so nodes with equal cost are expanded in FIFO order as stated before in **Nodes class**

***heapq*** implements a binary min-heap. It does not keep the whole list sorted. It only guarantees that the lowest-cost node is at the front, and that is the node popped next.

While frontier is not empty, the loop continues. Once it’s empty, if it became empty before a goal was found, it’ll return that no solution was found, along with extra parameters that will be explained later.

Considering the frontier still has values:

Get the next node to be expanded, which in the case of A* it’s the one with lowest cost.

Check, is it the goal? If yes, done! Return the board and the extra parameters.

If not the goal? Continue the loop.

If the board itself was already explored, skip the loop, and get the next board.

**Would there be two boards in the frontier of the same content and positions?**

Yes, different path can get same solution.

**Why did we decide to skip if the board was already explored?**

Because if it was already explored, already expanded, that means we'd *again* make the same moves. We don't need to re-do everything again.

If the board was not explored before, the loop continues.

Add the board in the explored.

Our main idea is moving the blank itself. So, we get the position of the blank.

There are a few illegal moves, if any of them occurred to the blank's conditions, we skip *these* movements and try another move. We used the priority: *Up, Down, Left, Right* according to the reference assigned by the end of the assignment. No certain priority should've been met.

***For example,***

If the blank is at the very top left corner, it can't go left nor up.

The condition restriction blank < 3 guarantee that positions 0 1 and 2 cannot go up. In that case, it can't go up.

The condition restriction blank > 5 ensure that positions 6 7 and 8 cannot go down, in my case of very top left corner, my blank can move down.

The constraint blank % 3 == 0 guarantee that the left corner cannot go any more left, and in this case, the blank cannot go left.

The final constraint, blank % 3 == 2 guarantee that the right corner cannot go any more right, in my case it can go right.

Therefore, my blank in that example can go down, and right.

On each iteration, the move is tested if it was illegal or not. If yes, skipped, if not, the blank is moved by the function **moveBlank**. moveBlank returns the new board, if the tuple was already in the explored then don't add.

I then make the board as a new Node, then add it to frontier and the loop continues until either the goal was found, or no goal was found, and the frontier is empty.

**Note**: I won't add a board that was already explored, but perhaps at that moment it wasn't yet explored, so it's added to the frontier. However, the frontier already had that board, but I wouldn't know. That's why at the beginning, I checked if it was already explored before expanding it again.

The Manhattan distance and the Euclidean distance use the *same* algorithm, the only difference is when we make a new Node, for Manhattan I use **HeuristicsManhattan** and for Euclidean I use **HeuristicsEuclidean**, and the same goes for when we used the heapq function, where we entered the heuristics as a parameter.

*Other than that, the algorithm is the same.*

***Comparing the number of nodes expanded and output paths, and report which heuristic is more admissible.***

Using a sample run of initial board: 8 1 3 4 0 2 7 6 5

*For Manhattan:*

Nodes expanded: 1369

Path cost: 22

Output Path: ['Left', 'Up', 'Right', 'Down', 'Left', 'Down', 'Right', 'Up', 'Left', 'Up', 'Right', 'Right', 'Down', 'Down', 'Left', 'Up', 'Up', 'Left', 'Down', 'Right', 'Up', 'Left']

*For Euclidean:*

Nodes expanded: 1865

Path cost: 22

Output Path: ['Left', 'Up', 'Right', 'Down', 'Left', 'Down', 'Right', 'Up', 'Left', 'Up', 'Right', 'Right', 'Down', 'Down', 'Left', 'Up', 'Up', 'Left', 'Down', 'Right', 'Up', 'Left']

For both, the output path and cost of path is the same, because they're both optimal. However, there's a significant difference in nodes expanded number. Even for the running time when the sample ran, for Euclidean it was 0.0409 s while for Manhattan it was 0.0261 s.

However, the nodes expanded may differ due to the tie-breaking rule.

Regarding the admissibility of both. Admissible is that the heuristic function never overestimates the true cost, which makes A* here optimal.

**Manhattan distance** is admissible. Each tile must move at least its horizontal plus vertical distance, and each move shifts only one tile by one square, so it can't overestimate.

**Euclidean distance** is also admissible. A straight line is the shortest possible distance, so it is never more than the Manhattan distance, which is itself never more than the true cost.

So, which is more admissible if they're both admissible? Manhattan is more *informed*, it dominates Euclidean. Because it is closer to the true cost, A* expands fewer nodes with it while still finding the same optimal path.

The sample runs prove this. The difference in nodes expanded is shown clearly.

## 5. Notes regarding all algorithms

Considering **b** is the branching factor (at most 4 moves in the 8-puzzle), **d** is the depth of the shallowest goal, and **m** is the maximum depth of the search.

| Algorithm | Time | Space | Optimal |
|---|---|---|---|
| BFS | O(b^d) | O(b^d) | Yes (all moves cost 1) |
| DFS | O(b^m) | O(b·m) | No |
| IDS | O(b^d) | O(b·d) | Yes |
| A* | O(b^d) worst case | O(b^d) | Yes, if the heuristic is admissible. |

- **BFS** explores every level up to the goal and keeps all of it in memory, so it is heavy on space.
- **DFS** only stores the current path, so its space is small. It can go very deep and find a long, non-optimal path.
- **IDS** gets the optimal answer of BFS with the small memory of DFS. Repeating the shallow levels costs little because the last level dominates.
- **A*** uses the heuristic to expand fewer nodes in practice. A better heuristic prunes more: Manhattan is better than Euclidean here, since it is closer to the true cost. The worst case is still exponential.

In all algorithms, they return a few things. Including: **goal_node, expanded, depth**

For the goal_node, it's simply the goal we initiated. The algorithm may return None if none was found.

For the expanded, we initialize at the beginning of algorithm that nodes_expanded = 0, and each time the node is added to explore, nodes_expanded will be incremented by 1, until the goal_node is sent back.

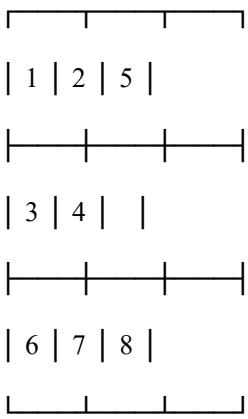
For the depth, our focus is the maximum depth, because searching depth is the maximum depth. At the beginning too, we initialize it by 0. Each time we get a board from the frontier, we check the maximum between the maximum_depth and the board's depth, whoever is more is assigned to the maximum depth. When the algorithm finishes once it returns the maximum_depth too.

## 6. Sample Runs

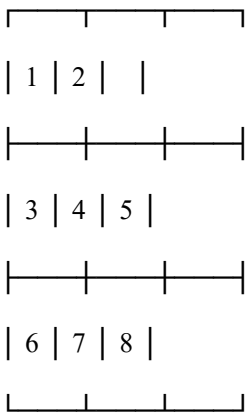
Our default is: 1,2,5,3,4,0,6,7,8

*A sample run:*

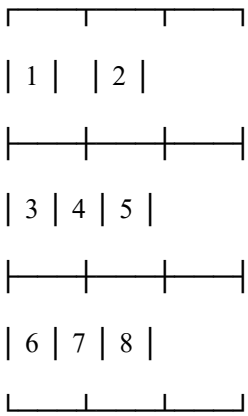
Initial board:



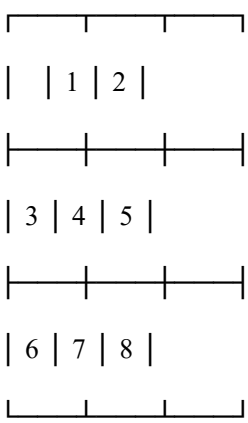

Step 1: move Up




Step 2: move Left



Step 3: move Left




Goal reached!

Path to goal: ['Up', 'Left', 'Left']

Cost of path: 3

Nodes expanded: 10

Search depth: 3

Running time: 0.0001 s